Tutorato 3 - Esercizi Extra

Matteo Mazzaretto

03 Novembre 2026



Extra 1: crocette

1. Dato `template <class T> T minimo(T a, T b)`, cosa succede con la chiamata `minimo(3, 4.5)`?
A) compila, `T` viene dedotto come `double`
B) errore di compilazione: `T` verrebbe dedotto sia `int` sia `double`
C) compila, `T` è `int` e il valore `4.5` viene troncato

2. Perché la definizione di un template di funzione si mette nel file header?
A) perché il compilatore deve vedere la definizione per generare l'istanza dove il template viene usato
B) perché i template di funzione sono sempre `inline` per standard
C) perché nei file `.cpp` i template non possono avere parametri di tipo

Extra 1: soluzione

1 → B. Nella deduzione implicita i parametri dello stesso tipo `T` devono essere **esattamente** uguali e non si applicano conversioni come `int` → `double`. Si risolve con l'istanziazione esplicita: `minimo<double>(3, 4.5)`.

2 → A. La sola definizione del template non produce codice: l'istanza viene generata quando il template è usato, e per farlo il compilatore deve vedere la definizione. Per questo non si può fare information hiding con i template.

Extra 2: cosa stampa?

UNIVERSITÀ DI PADOVA

```
template <class T>
T minimo(T a, T b){ return a < b ? a : b; }

template <class T, int size>
int dim(T (&a)[size]){ return size; }

int main(){
  int i = 3; double d = 2.5;
  int ia[7]; char ca[4];

  cout << minimo(i, 8) << " ";
  cout << minimo<double>(i, d) << " ";
  cout << minimo<int>(d, i) << " ";
  cout << dim(ia) + dim(ca);
}
```

Extra 2: soluzione

Output:

```
3 2.5 2 11
```

- `minimo(i, 8)`: `T` viene dedotto `int` da entrambi gli argomenti, il minimo è `3`
- `minimo<double>(i, d)`: istanziazione esplicita, `i` diventa `3.0`; il minimo è `2.5`
- `minimo<int>(d, i)`: questa volta è `d` a essere convertito in `int`, diventa `2`; il minimo è `2`
- `dim(ia)` e `dim(ca)`: il parametro valore `size` viene dedotto dalla dimensione dell'array, quindi `7 + 4 = 11`

Extra 3: scrivi la funzione

Scrivi il template di funzione `cerca` che, dati un array statico (di qualsiasi tipo e dimensione) e un valore, restituisce l'indice della prima occorrenza del valore, oppure `-1` se non è presente. La dimensione dell'array **non** va passata come parametro.

```
int main(){
  int ia[5] = {3, 8, 1, 8, 5};
  char ca[3] = {'x', 'y', 'z'};
  cout << cerca(ia, 8) << " ";
  cout << cerca(ca, 'q') << " ";
  cout << cerca(ca, 'z');
}
// output: 1 -1 2
```

Extra 3: soluzione

```
template <class T, int size>
int cerca(const T (&a)[size], const
T& x){
  for(int i = 0; i < size; i++)
    if(a[i] == x) return i;
  return -1;
}
```

- `size` è un parametro valore, dedotto dalla dimensione dell'array
- L'array va passato per **riferimento**: per valore decadrebbe in puntatore e la dimensione andrebbe persa
- `const T (&a)[size]` perché l'array non viene modificato
- Serve che `T` abbia `operator==`

Extra 4: cosa stampa?

```
template <class T>
class Cont{
  public:
    static int n;
    Cont(){ n++; }
    ~Cont(){ n--; }
};

template <class T>
int Cont<T>::n = 0;

int main(){
  Cont<int> a, b;
  Cont<char> c;
  {
    Cont<int> d;
    cout << Cont<int>::n << " ";
  }
  cout << Cont<int>::n << " " << Cont<char>::n << " ";
  Cont<double> e;
  cout << Cont<double>::n;
}
```

Extra 4: soluzione

Output:

```
3 2 1 1
```

- Un membro statico di un template di classe esiste **per ogni istanza**: `Cont<int>::n`, `Cont<char>::n` e `Cont<double>::n` sono tre variabili distinte
- `a` e `b` portano `Cont<int>::n` a 2; `c` porta `Cont<char>::n` a 1
- `d` nel blocco: `Cont<int>::n` vale 3, e viene stampato `3`
- All'uscita dal blocco `d` viene distrutto: `Cont<int>::n` torna a 2, mentre `Cont<char>::n` vale ancora 1
- `e` è la prima istanza di `Cont<double>`: il suo contatore parte da 0 e arriva a 1

Extra 5: crocette

1. `Queue<int>` e `Queue<string>` sono due istanze dello stesso template di classe. Come sono tra loro?
A) la stessa classe, con membri di tipo diverso
B) due classi distinte, che non possono accedere l'una alla parte privata dell'altra
C) due classi distinte, ma amiche tra loro per default

2. Quando viene istanziato un metodo di un template di classe?
A) insieme alla classe, tutti i metodi contemporaneamente
B) solo se e quando il programma usa effettivamente quel metodo
C) all'avvio del programma, a run-time

Extra 5: soluzione

1 → B. Ogni istanza è una classe completamente distinta: per vedere la parte privata di un'altra istanza servirebbe una dichiarazione `friend` esplicita.

2 → B. Istanziare la classe non istanzia i suoi metodi: ognuno viene generato solo se il programma lo usa. Tutto avviene a compile-time.

Extra 6: cosa stampa?

```
template <class T, int N = 2>
class Buf{
  public:
    T v[N];
    Buf(){ cout << "B" << N << " "; }
    ~Buf(){ cout << "~B" << N << " "; }
    int size() const { return N; }
};

int main(){
  Buf<int> a;
  Buf<char, 3> b;
  Buf<int, 2> c;
  cout << a.size() + b.size() + c.size() << " ";
}
```

Extra 6: soluzione

Output:

```
B2 B3 B2 7 ~B2 ~B3 ~B2
```

- `Buf<int>` usa il valore di default `N = 2`: stampa `B2`
- `Buf<char, 3>` stampa `B3`
- `Buf<int>` e `Buf<int, 2>` sono la **stessa** istanza del template
- `2 + 3 + 2 = 7`
- Alla fine del `main` gli oggetti si distruggono in ordine inverso: `c`, `b`, `a`, cioè `~B2 ~B3 ~B2`

Extra 7: scrivi i metodi

Scrivi le definizioni **esterne** di `Queue<T>::add` e `Queue<T>::remove`. Puoi assumere che `remove` venga chiamata solo su una coda non vuota, e ignora il distruttore.

```
template <class T> class QueueItem;

template <class T>
class Queue{
  public:
    Queue(): primo(0), ultimo(0) {}
    bool is_empty() const { return primo == 0; }
    void add(const T&);  // inserisce in coda
    T remove();          // rimuove dalla testa e restituisce l'elemento
  private:
    QueueItem<T>* primo;
    QueueItem<T>* ultimo;
};

template <class T>
class QueueItem{
  public:
    QueueItem(const T& t): info(t), next(0) {}
    T info;
    QueueItem* next;
};
```

Extra 7: soluzione

```
template <class T>
void Queue<T>::add(const T& x){
  QueueItem<T>* q = new QueueItem<T>(x);
  if(is_empty()) primo = ultimo = q;
  else {
    ultimo->next = q;
    ultimo = q;
  }
}

template <class T>
T Queue<T>::remove(){
  QueueItem<T>* q = primo;
  T aux = q->info;
  primo = primo->next;
  if(!primo) ultimo = 0;
  delete q;
  return aux;
}
```

- Ogni definizione esterna ripete `template <class T>` e usa `Queue<T>::`
- `add`: se la coda è vuota il nuovo nodo è sia `primo` sia `ultimo`, altrimenti si aggancia dopo `ultimo`
- `remove`: si copia `info` prima di fare `delete` del nodo
- Se dopo la rimozione la coda è vuota, `ultimo` va azzerato, altrimenti resta un puntatore a un nodo cancellato

Extra 8: cosa stampa?

Ricorda che un `bool` stampato con `cout` vale `0` o `1`.

```
int main(){
  vector<int> v(3, 4);
  v.push_back(9);
  v.push_back(1);
  v.pop_back();
  cout << v.size() << " " << (v.capacity() >= v.size()) << " ";
  cout << v.front() << " " << v.back() << " ";

  vector<int>::iterator it = v.begin();
  ++it;
  *it = 7;
  int s = 0;
  for(it = v.begin(); it != v.end(); ++it) s += *it;
  cout << s;
}
```

Extra 8: soluzione

Output:

```
4 1 4 9 24
```

Il vector passa da `4 4 4` a `4 4 4 9 1`, poi `4 4 4 9`, infine `4 7 4 9`

- `vector<int> v(3, 4)` crea 3 elementi uguali a 4
- `push_back(9)` e `push_back(1)` aggiungono in coda, `pop_back()` toglie l'1: `size()` vale 4
- `capacity() >= size()` è sempre vero: `1`
- `front()` è `4`, `back()` è `9`
- `++it` punta al secondo elemento, che diventa `7`
- La somma dei valori finali è `4 + 7 + 4 + 9 = 24`

Extra 9: crocette

1. In un template, la riga `T::const_iterator it = c.begin();` non compila. Cosa manca?
A) `typename` davanti a `T::const_iterator`, perché è un nome dipendente che denota un tipo
B) `template` prima di `T`
C) `static` davanti alla dichiarazione

2. Quale di queste operazioni su un `vector` ha costo lineare?
A) l'accesso `v[i]`
B) `push_back`, in costo ammortizzato
C) l'inserimento con `insert` in testa

Extra 9: soluzione

1 → A. Dentro un template il compilatore assume che `T::const_iterator` sia un **valore**. Per dire che è un tipo serve `typename T::const_iterator it = c.begin();`.

2 → C. Un `vector` è un array dinamico: l'accesso casuale è `O(1)`, `push_back` ha costo ammortizzato costante, ma inserire in testa obbliga a spostare tutti gli elementi, con costo `O(n)`.

Extra 10: cosa stampa?

`list` è una lista doppiamente linkata: `push_front` e `push_back` inseriscono rispettivamente in testa e in coda.

```
template <class C>
int somma_pari(const C& c){
  int s = 0;
  for(typename C::const_iterator it = c.begin(); it != c.end(); ++it)
    if(*it % 2 == 0) s += *it;
  return s;
}

int main(){
  vector<int> v;
  for(int i = 1; i <= 6; i++) v.push_back(i * i);

  list<int> l;
  l.push_back(3); l.push_front(8);
  l.push_back(10); l.push_front(5);

  cout << somma_pari(v) << " " << somma_pari(l) << " ";
  cout << l.front() << " " << l.back();
}
```

Extra 10: soluzione

Output:

`56 18 5 10`

`l` contiene `5 8 3 10`

- `typename C::const_iterator` è necessario perché `C::const_iterator` è un nome dipendente
- `v` contiene `1 4 9 16 25 36`: i pari sono `4 + 16 + 36 = 56`
- `l`: `3`, poi `8` in testa, `10` in coda e `5` in testa, quindi `5 8 3 10`
- I pari di `l` sono `8 + 10 = 18`
- `l.front()` è `5` e `l.back()` è `10`

Extra 11: scrivi la funzione

Scrivi il template di funzione `conta` che, dato un contenitore STL qualsiasi e un valore, restituisce quante volte il valore compare nel contenitore. Il `main` deve funzionare con `vector<int>` e con `list<char>`.

```
int main(){
  vector<int> v;
  v.push_back(4); v.push_back(7); v.push_back(4);
  list<char> l;
  l.push_back('a'); l.push_back('b');

  cout << conta(v, 4) << " ";
  cout << conta(l, 'z');
}
// output: 2 0
```

Extra 11: soluzione

```
template <class C, class T>
int conta(const C& c, const T& x){
  int n = 0;
  for(typename C::const_iterator it =
c.begin();
      it != c.end(); ++it)
    if(*it == x) n++;
  return n;
}
```

- Due parametri di tipo: `C` (il contenitore) e `T` (il valore cercato), dedotti dagli argomenti
- `typename` è obbligatorio: `C::const_iterator` è un nome dipendente che denota un tipo
- `c` è `const`, quindi si usa `const_iterator`
- Si scorre l'intervallo `[begin, end)` e `end()` non va dereferenziato

Extra 12: domande d'esame

1. Una classe annidata (inner class):
A) È un membro della classe che la contiene.
B) È automaticamente amica della classe esterna.
C) Non può accedere ai membri privati della classe esterna
D) Può essere dichiarata solo private.
E) Non può avere metodi statici.

2. Un template di classe:
A) Può avere solo parametri di tipo.
B) Genera codice solo quando viene istanziato.
C) Viene istanziato sempre automaticamente.
D) È equivalente a una macro del preprocessore.

Extra 12: soluzione

1 → A. Una classe annidata è un membro della classe che la contiene.

- B: falsa, non è `friend` della classe esterna: l'amicizia va dichiarata esplicitamente
- C: falsa, essendo un membro può accedere ai membri privati della classe esterna (non vale il contrario)
- D: falsa, può essere `public`, `protected` o `private`
- E: falsa, può avere membri e metodi statici

2 → B. Un template di classe è solo uno schema: il compilatore genera codice quando viene istanziato.

- A: falsa, esistono anche i parametri valore (es. `template <class T, int N>`)
- C: falsa, viene istanziato solo ciò che il programma usa (anche i singoli metodi, solo se usati)
- D: falsa, il compilatore controlla i tipi, non è una sostituzione testuale come una macro